

1 Introduction

RNN, 循环神经网络

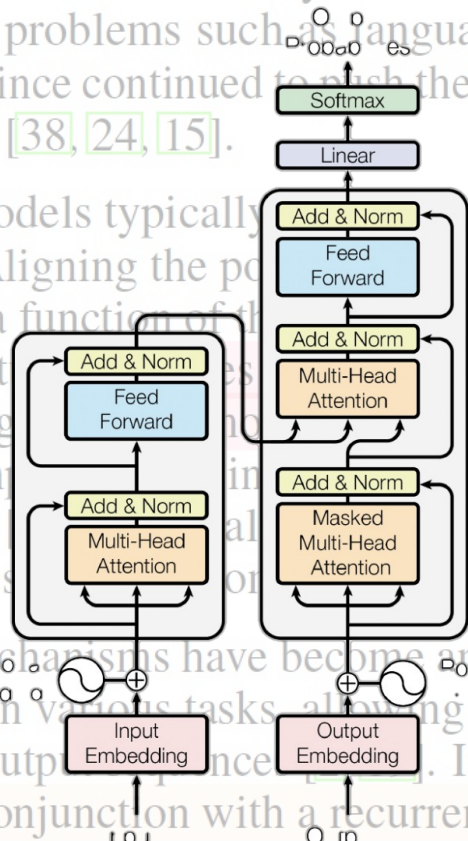
LSTM, 长短期记忆网络, 属于 RNN

GRU, 门控循环单元, 属于 RNN

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [25, 2, 11]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15].

Recurrent models typically process sequences sequentially. Aligning the positions of the input and output states h_t , as a function of the sequence length, is a non-trivial task. This inherent sequential nature of recurrent models is a significant impediment to parallelization and efficient computation. Attention mechanisms have become an integral part of modern sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequence [27]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture that eschews recurrence, instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.



人话版

《Attention is all you need》论文解读

Transformer

架构介绍

RNN及其变体
是当前序列转导任务的
主流方法

RNN的局限性
根本原因:
设计每一层训练过程
都有并行化

注意力机制的引入:
能够捕捉序列中任意位置
的依赖关系, 而不依赖于
递归计算的顺序性,
但是, 在绝大多数情况下,
它仅仅作为一种增强,
仍然与RNN一同使用

本文提出了 Transformer
模型, 它完全依赖于注意
力机制, 不再基于递归网
络RNN

主线：创新为何能取得革命性突破？

- 《Attention is All You Need》核心创新点：Transformer 架构和 Self-Attention 自注意力机制

思路

1. 以往模型的结构和处理逻辑，它们存在的问题 / 瓶颈——结构为何会导致瓶颈？
 - 涉及 1、2 节：Introduction & Background
2. 论文中创新性提出的模型结构，这样的结构解决了什么问题，为什么可以解决？
 - 核心：Transformer 架构和 Self-Attention 自注意力机制
 - 涉及 3、4 节：Model Architecture & Why Self-Attention

论文地址

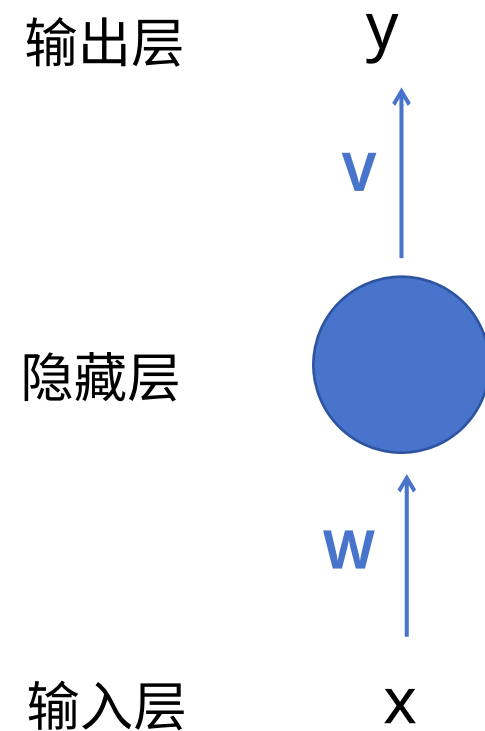
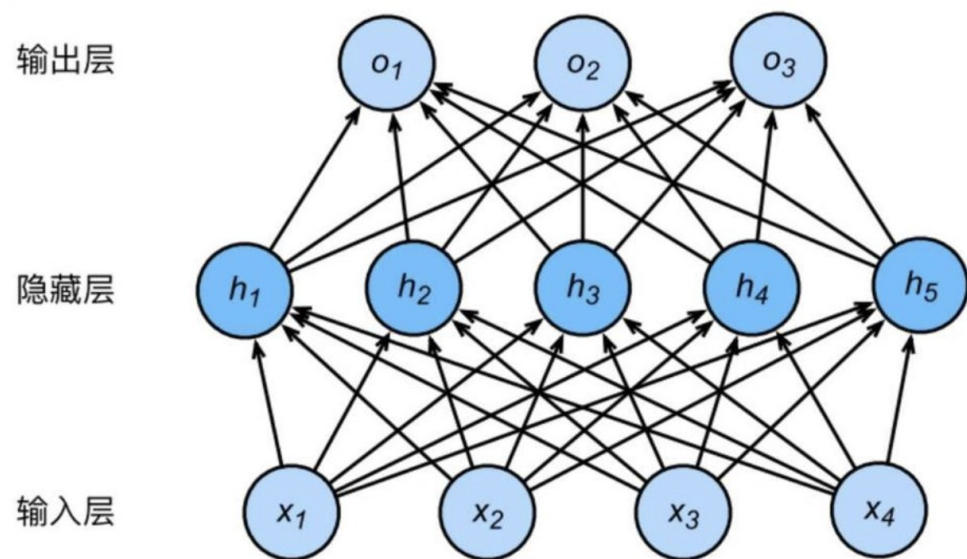
<https://arxiv.org/abs/1706.03762>

Sequence Transduction Model 序列转导模型

什么是序列转导模型（ Sequence Transduction Model ）

- —— 是处理序列数据的模型
- 序列数据：具有顺序关系的数据，每个元素的顺序对于数据的整体含义非常重要
- 典型的序列转导任务
 - **文本翻译**：翻译前的文本 ("How are you") -> 翻译后的文本（“你好吗？”）；
 - **文本生成**：用户的输入文本 ("How are you?") -> AI 生成的回复文本 ("I'm fine, thank you.")；
 - **语音转文字**：一个音频片段 -> 音频的文字转写；

Feedforward Neural Network (FNN) 前馈神经网络



Feedforward Neural Network (FNN) 前馈神经网络

为什么不适合序列转导任务?

输入：水是有毒的

步骤一：分词 (Tokenization)

["水", "是", "有毒", "的"]

步骤二：词向量表示 (Embedding)

"水" $\rightarrow [0.2, -0.1, 0.3, 0.0]$

"是" $\rightarrow [0.0, 0.5, -0.2, 0.1]$

"有毒" $\rightarrow [0.9, -0.3, 0.4, 0.2]$

"的" $\rightarrow [0.1, 0.0, 0.0, 0.1]$

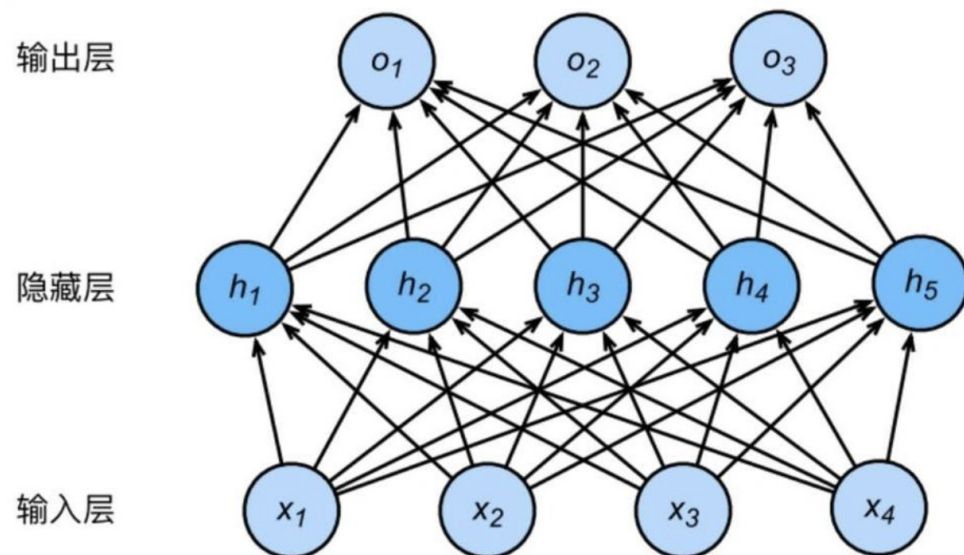
步骤三：合并词向量 (平均或拼接)

平均

- 完全丢掉了词语的顺序

拼接

- FNN 需要固定维度的输入，对不同长度的句子处理效率低下
- 仍然会将句子视作一个整体来处理，无法理解真正的“谁先谁后”的关系；



Recurrent Neural Network (RNN) 循环神经网络

RNN 解决的问题：

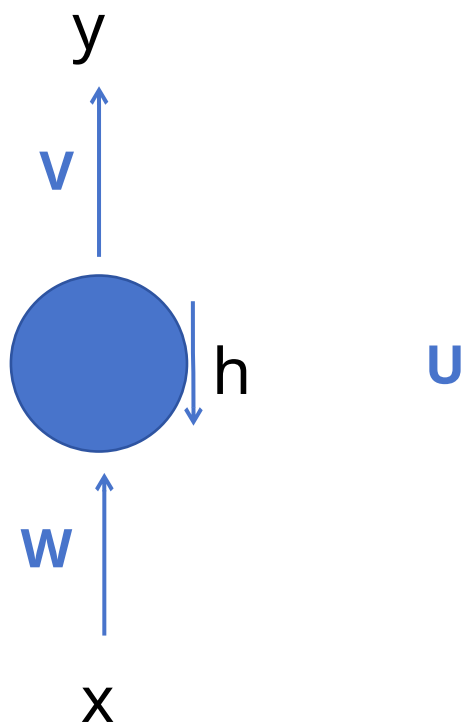
- 能够建模词序：RNN 是按时间顺序（token 顺序）逐个处理输入的；
- 能够建模上下文依赖：RNN 是逐个喂入词语，并且会有“记忆”机制；
- 支持不定长输入：不再需要 FNN 那种固定长度的输入格式，句子多长都行；

Recurrent Neural Network (RNN) 循环神经网络

输出层

隐藏层

输入层



$$h_t = g(Wx_t + Uh_{t-1})$$

$$y_t = g(Vh_t)$$

x_t : t 时间步的输入

h_t : t 时间步的状态

y_t : t 时间步的输出（也可能没有）

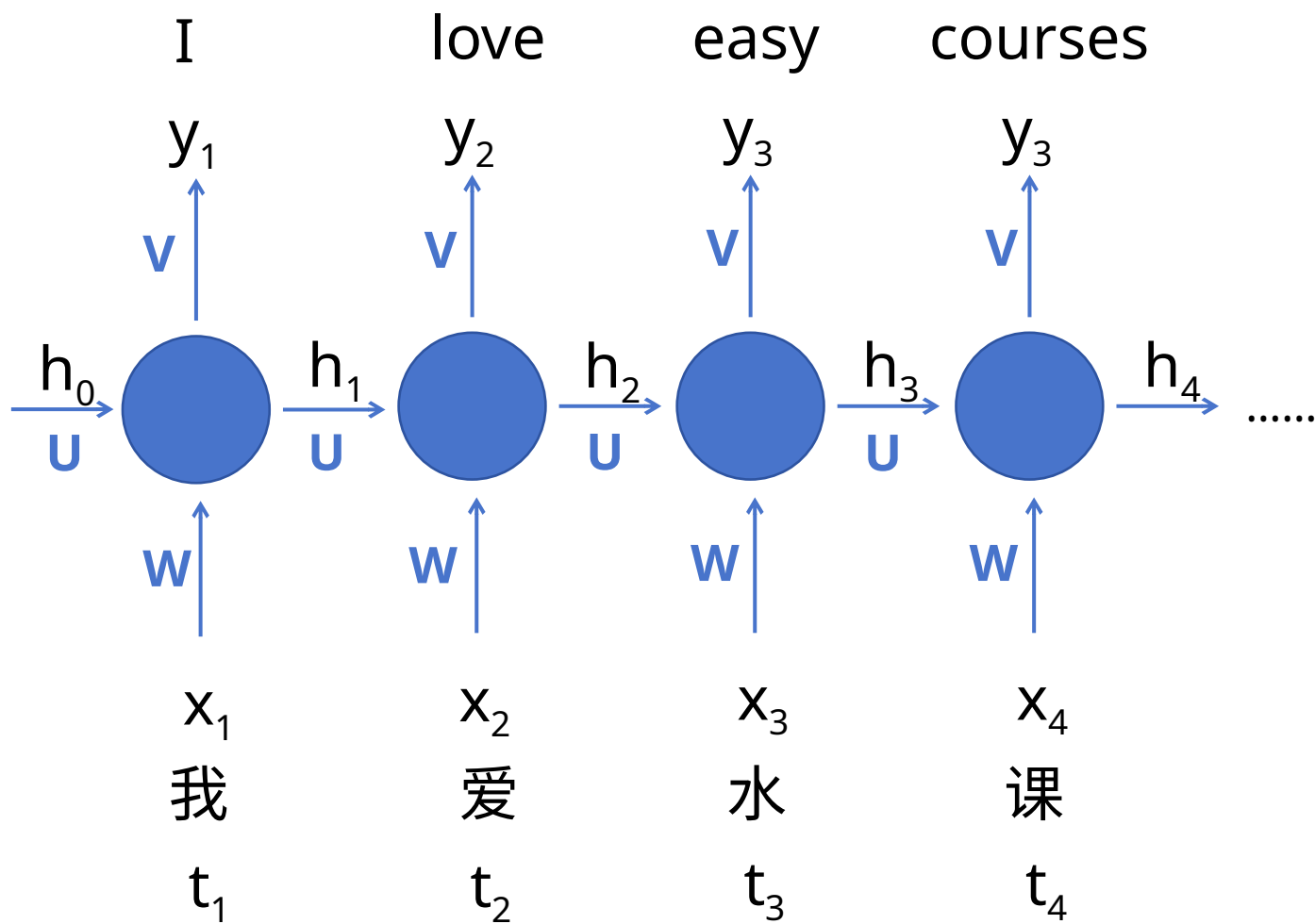
U, V, W : 权重矩阵

h_t 中的 g : 激活函数，如

ReLU, sigmoid, 用于引入非线性，增强表达能力；同时限制数值范围，避免梯度爆炸 / 消失。

y_t 中的 g : 视任务而定，比如如果是分类问题，可能是 softmax 函数，用于把输出转成概率分布。

Recurrent Neural Network (RNN) 循环神经网络



$$h_t = g(Wx_t + Uh_{t-1})$$

$$y_t = g(Vh_t)$$

x_t : t 时间步的输入

h_t : t 时间步的状态

y_t : t 时间步的输出 (也可能没有)

U, V, W : 权重矩阵

h_t 中的 g : 激活函数, 如

ReLU, sigmoid, 用于引入非线性, 增强表达能力; 同时限制数值范围, 避免梯度爆炸 / 消失。

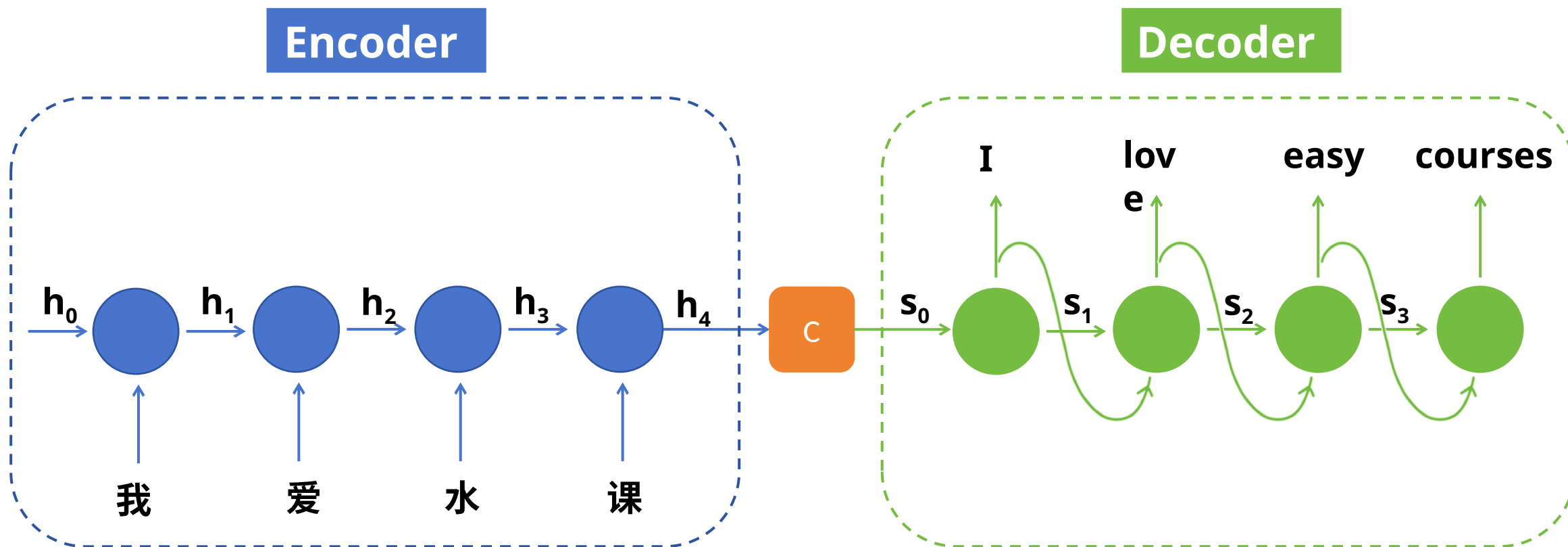
y_t 中的 g : 视任务而定, 比如如果是分类问题, 可能是 softmax 函数, 用于把输出转成概率分布。

Encoder and Decoder 编码器 - 解码器结构

编码器 - 解码器结构解决的问题

- 支持输入输出不等长

Encoder and Decoder 编码器 - 解码器结构

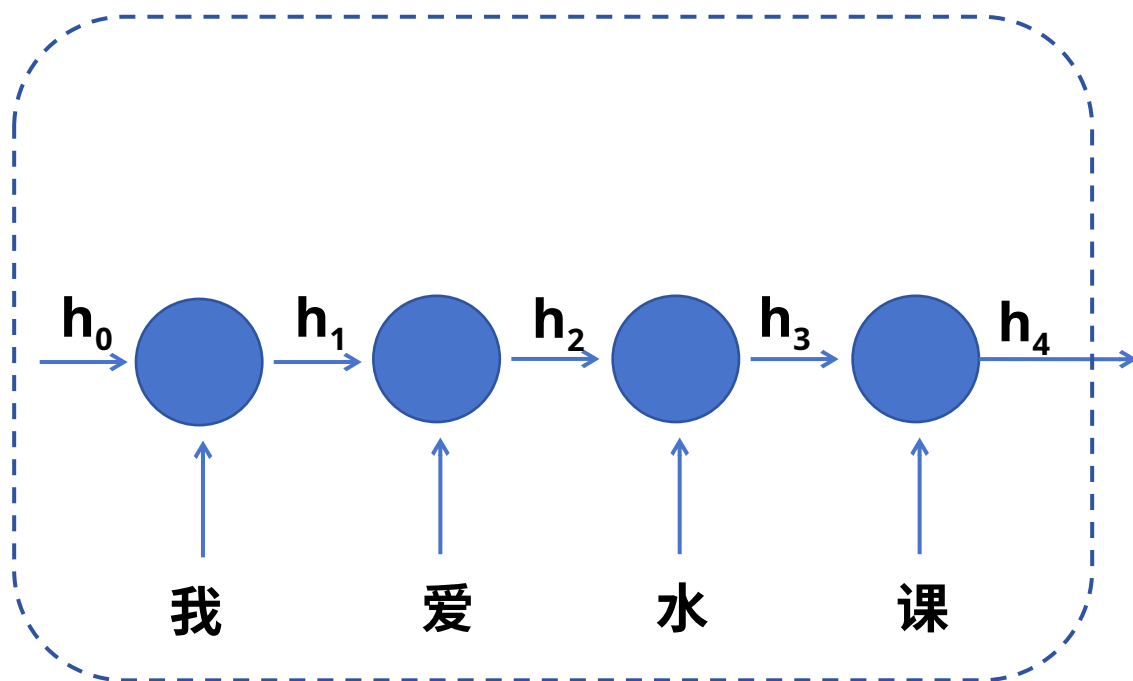


C：“上下文向量（context vector）”

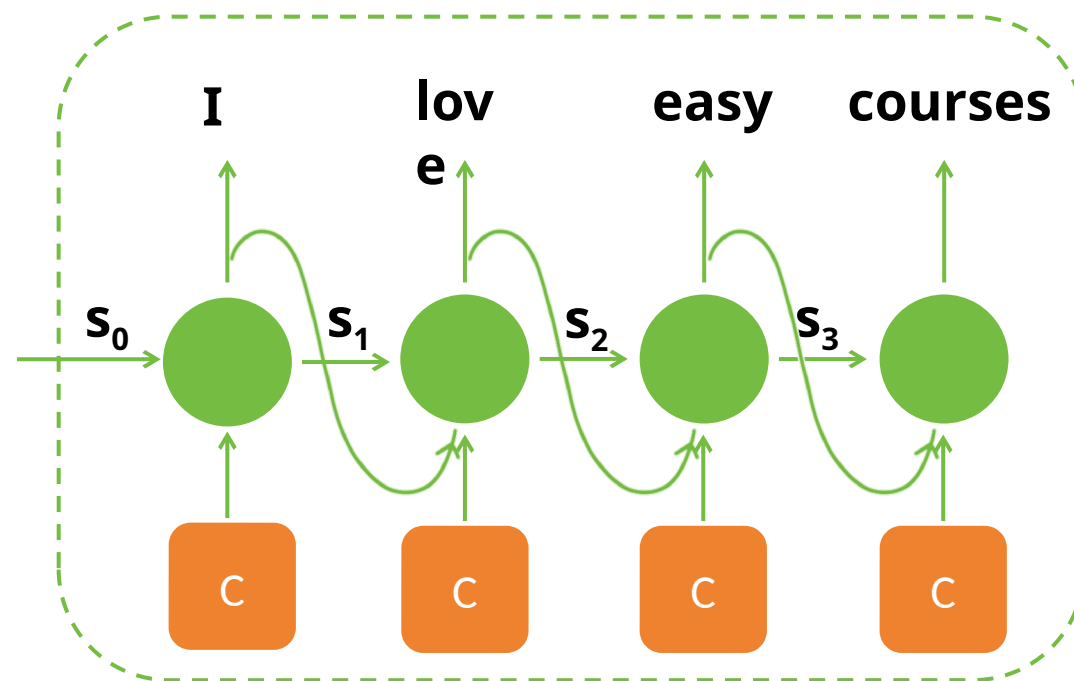
- 它是对整个输入序列的语义编码，是一个固定长度的向量，涵盖了整个输入文本的语义信息。
- 最简单的编码方式： $C =$ 最后一个时间步的隐藏状态输出 (h_4)
- 它将作为 Decoder 的输入，用于生成目标序列。

Encoder and Decoder 编码器 - 解码器结构

Encoder



Decoder



C: “上下文向量 (context vector)”

- 它是对整个输入序列的语义编码，是一个固定长度的向量，涵盖了整个输入文本的语义信息。
- 最简单的编码方式：C = 最后一个时间步的隐藏状态输出 (h_4)。
- 它将作为 Decoder 的输入，用于生成目标序列。

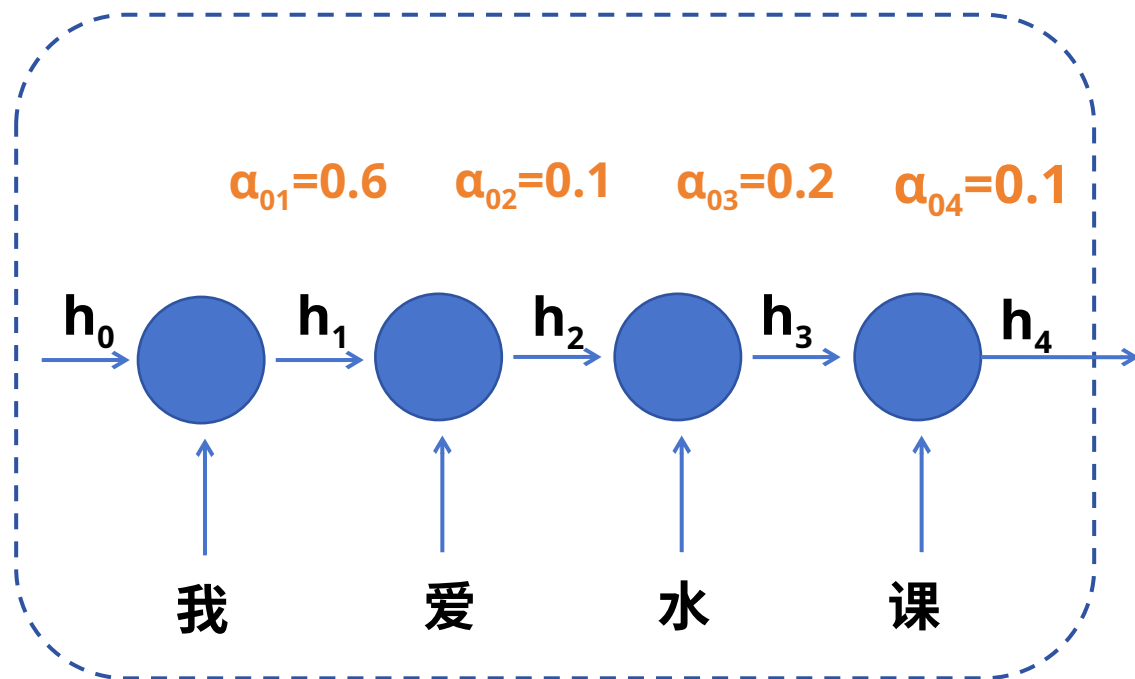
Attention Mechanism 注意力机制

注意力机制解决的问题：

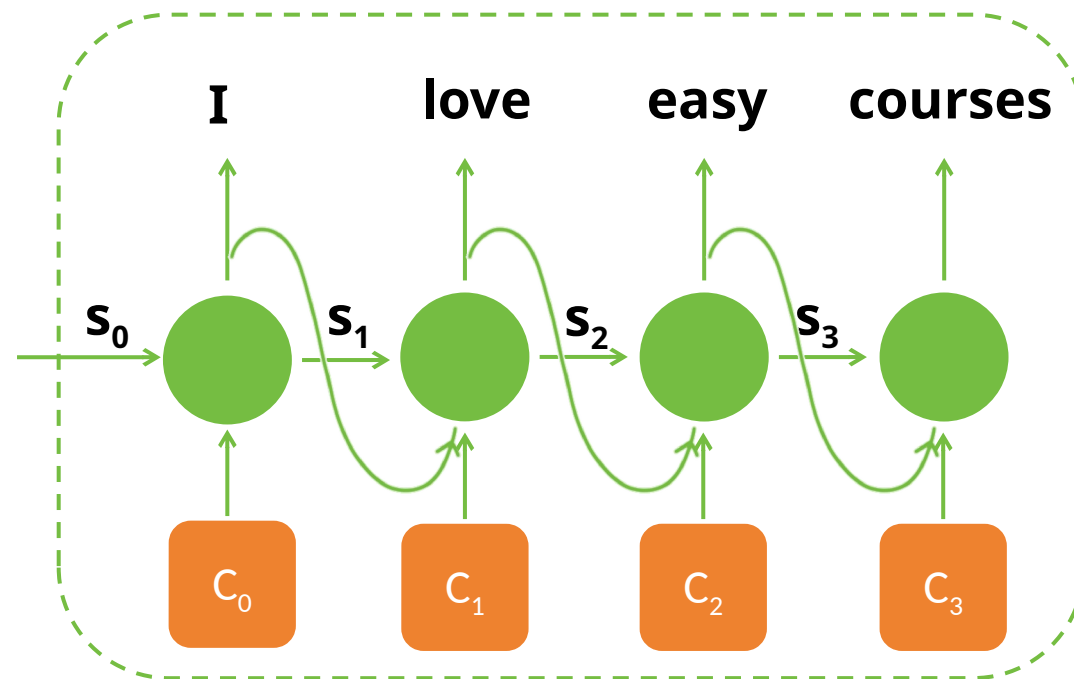
1. 解决模型处理长序列时的“遗忘”问题：随着序列长度的增长，远距离依赖信息在传递过程中易被稀释，导致模型对长距离依赖关系的建模能力减弱。
2. 解决不同时间步输入对当前时刻输出的“重要性”问题：所有时间步的输入在计算当前时刻输出时被同等对待，忽略了不同时间步对当前时刻输出的重要性可能存在的差异。

Attention Mechanism 注意力机制

Encoder



Decoder

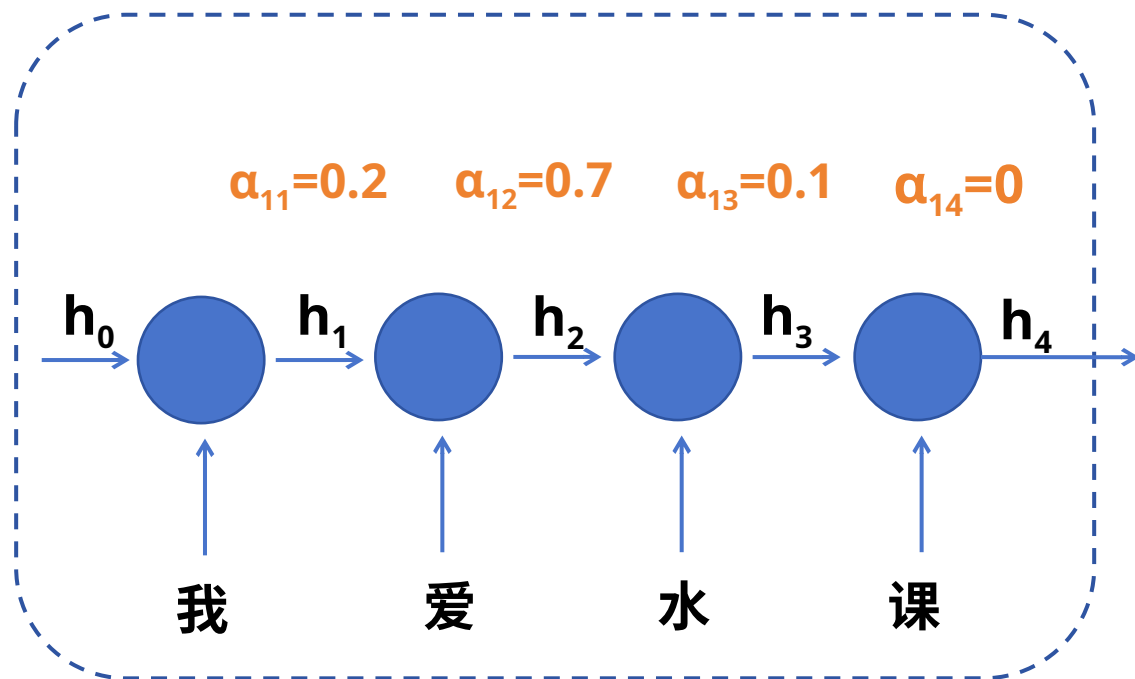


$$C_i = \sum_{j=1}^n \alpha_{ij} h_j$$

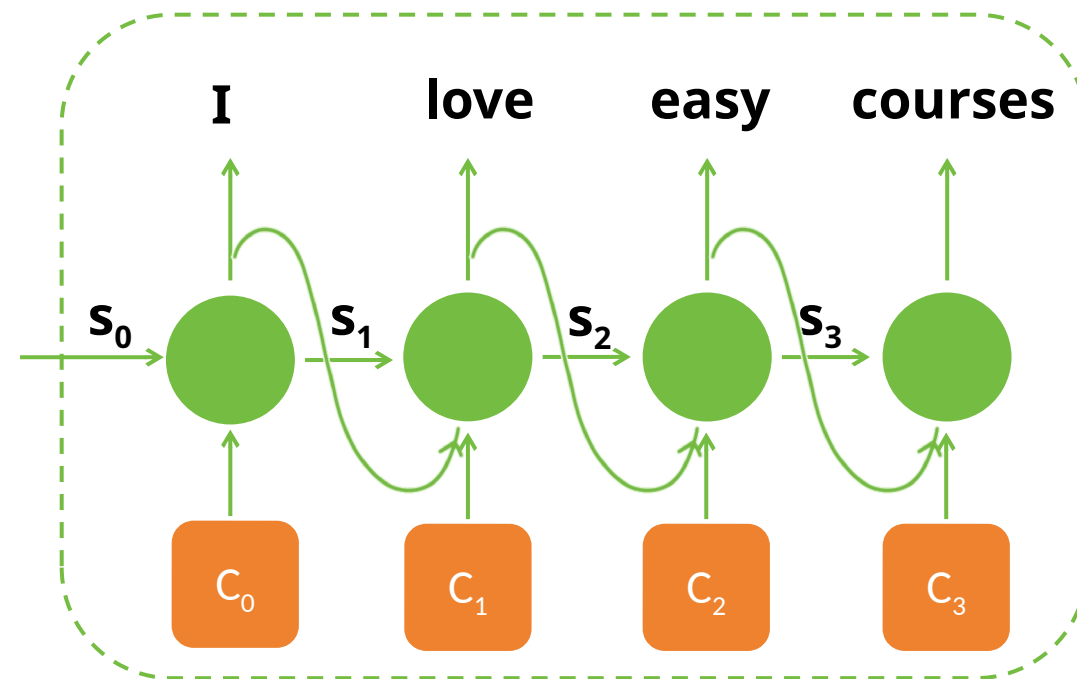
$$C_0 = 0.6h_1 + 0.1h_2 + 0.2h_3 + 0.1h_4$$

Attention Mechanism 注意力机制

Encoder



Decoder

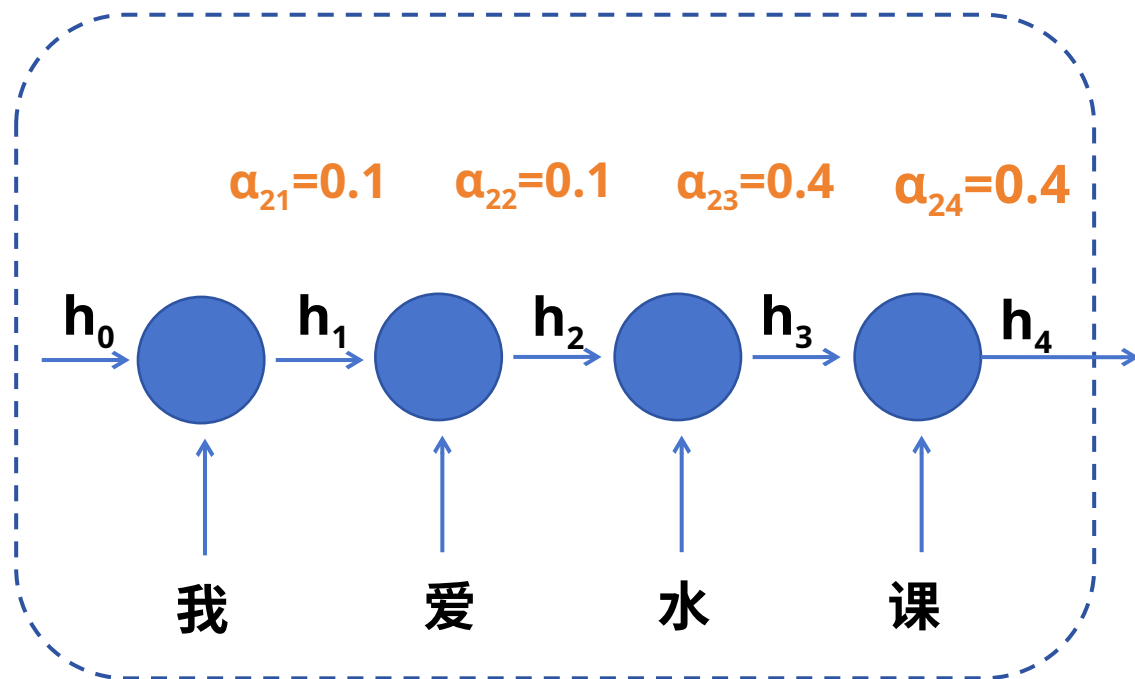


$$C_i = \sum_{j=1}^n \alpha_{ij} h_j$$

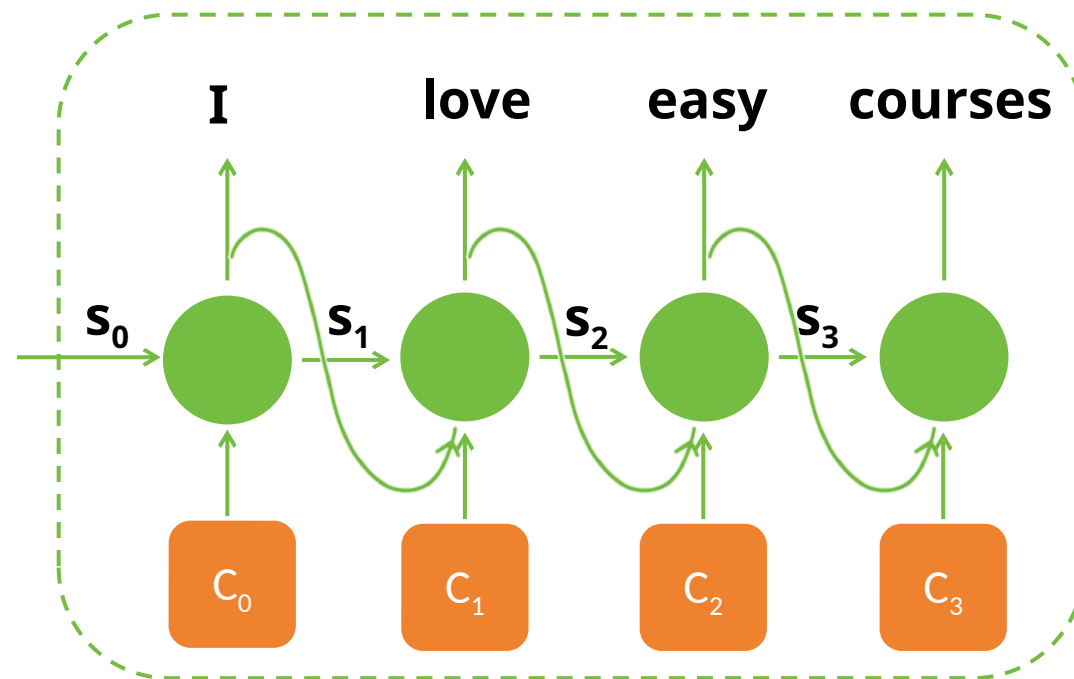
$$C_1 = 0.2h_1 + 0.7h_2 + 0.1h_3 + 0h_4$$

Attention Mechanism 注意力机制

Encoder



Decoder



$$C_i = \sum_{j=1}^n \alpha_{ij} h_j$$

$$C_2 = 0.1h_1 + 0.1h_2 + 0.4h_3 + 0.4h_4$$

还有什么问题没被解决？

- —— 串行化计算

Transformer & Self-Attention

- 能够建模词序
- 能够建模上下文依赖 RNN
- 支持不定长输入
- 支持输入输出不等长 编码器 - 解码器结构
- 解决处理长序列时的“遗忘”问题
- 解决不同时间步输入对当前时刻输出的“重要性”问题
- 解决串行化计算问题

注意力机制

Transformer & Self-Attention

Introduction & Background 思路

RNN：顺序计算的瓶颈

传统模型：RNN / LSTM / GRU

↓
逐步生成 hidden state

↓
优点：适合建模序列（如语言）
缺点：计算完全串行，无法并行；
长序列计算慢，难训练；梯度消失。

↓
引入 Attention 注意力机制
缓解信息瓶颈

RNN + Attention

↓
加入注意力模块，聚焦重要输入位置

↓
优点：改善长距离依赖建模能力
缺点：依然使用 RNN，还是串行计算

Introduction

为解决 RNN 顺序计算问题的研究：使用 CNN 尝试并行化

Extended Neural GPU / ByteNet / ConvS2S

↓
将 CNN 作为编码器，支持输入并行

↓
存在问题：信息传递距离仍受限制（ConvS2S 是线性，ByteNet 是对数级），远距离依赖建模仍困难

↓
Memory Network：注意力机制替代 RNN 的递归

End-to-End Memory Network

↓
不再依赖 RNN，使用多轮注意力从“记忆”中读取信息

↓
存在问题：仍然有多轮递归过程（multi-hop attention），并不高效

↓
Transformer：抛弃 RNN 和 CNN，只用 Self-Attention 自注意力机制

Transformer 架构

↓
完全基于 Self-Attention，输入每个位置彼此之间做注意力（ $Q \cdot K$ ），可以完全并行训练所有位置；
多头注意力（Multi-Head Attention）提升表达能力

↓
优点：并行训练高效；建模远距离依赖只需一次注意力操作（ $O(1)$ ）；
在翻译、语言建模等任务上刷新 SOTA

Background

三种不同的注意力机制 Attention Mechanism

特性	RNN+Attention	Memory Network	Transformer (Self-Attention)
是否用 RNN	✓ 是	✓ 可用	✗ 无
注意力对象	编码器隐藏状态（输入序列）	外部记忆块（可多轮）	自身序列内部所有位置
是否递归	✗ 否	✓ 是，多轮推理	✗ 否，一次性计算
是否可并行	✗ 不可	✗ 多数不可	✓ 完全并行
是否用于 encoder	✗ 否，只用于 decoder	一般用于推理	✓ 是，encoder/decoder 都用

- RNN 引入注意力机制是为了弥补信息瓶颈，但仍是顺序处理；
- Memory Network 把注意力用在“记忆检索”上，强调推理；但仍然有多轮递归过程，并不高效；
- Transformer 的自注意力机制是“全局互相关注”，完全摆脱了序列结构，支持并行，是最底层的范式改变。